

std::list

Defined in header <list>

```
template<
    class T,
    class Allocator = std::allocator<T>
> class list;
(1)
```

```
namespace pmr {
    template< class T >
        using list = std::list<T, std::pmr::polymorphic_allocator<T>>;
    (2) (since C++17)
}
```

std::list is a container that supports constant time insertion and removal of elements from anywhere in the container. Fast random access is not supported. It is usually implemented as a doubly-linked list. Compared to std::forward_list this container provides bidirectional iteration capability while being less space efficient.

Adding, removing and moving the elements within the list or across several lists does not invalidate the iterators or references. An iterator is invalidated only when the corresponding element is deleted.

std::list meets the requirements of *Container*, *AllocatorAwareContainer*, *SequenceContainer* and *ReversibleContainer*.

All member functions of std::list are **constexpr**: it is possible to create and use std::list objects in the evaluation of a constant expression.

However, defining a **constexpr** std::list variable is generally an error, because constant evaluation requires any dynamically allocated storage to be released in the same evaluation, which is usually not the case with std::list's initializer. (since C++26)

Template parameters

T - The type of the elements.									
T must meet the requirements of <i>CopyConstructible</i>. T must meet the requirements of <i>CopyAssignable</i> if <code>list::operator=</code> or <code>list::assign</code> is instantiated with T. (until C++11)									
The requirements that are imposed on the elements depend on the actual operations performed on the container. Generally, it is required that element type is a complete type and meets the requirements of <i>Erasable</i>, but many member functions impose stricter requirements. (since C++11) (until C++17)									
The requirements that are imposed on the elements depend on the actual operations performed on the container. Generally, it is required that element type meets the requirements of <i>Erasable</i>, but many member functions impose stricter requirements. This container (but not its members) can be instantiated with an incomplete element type if the allocator satisfies the allocator completeness requirements. (since C++17)									
<table><tr><th>Feature-test macro</th><th>Value</th><th>Std</th><th>Feature</th></tr><tr><td><code>__cpp_lib_incomplete_container_elements</code></td><td>201505L</td><td>(C++17)</td><td>Minimal incomplete type support</td></tr></table>	Feature-test macro	Value	Std	Feature	<code>__cpp_lib_incomplete_container_elements</code>	201505L	(C++17)	Minimal incomplete type support	
Feature-test macro	Value	Std	Feature						
<code>__cpp_lib_incomplete_container_elements</code>	201505L	(C++17)	Minimal incomplete type support						

Allocator - An allocator that is used to acquire/release memory and to construct/destroy the elements in that memory. The type must meet the requirements of *Allocator*. The behavior is undefined (until C++20) The program is ill-formed (since C++20) if Allocator::value_type is not the same as T.

Member types

Member type	Definition
value_type	T
allocator_type	Allocator
size_type	Unsigned integer type (usually std::size_t)
difference_type	Signed integer type (usually std::ptrdiff_t)
reference	value_type&
const_reference	const value_type&
pointer	Allocator::pointer (until C++11) std::allocator_traits<Allocator>::pointer (since C++11)
const_pointer	Allocator::const_pointer (until C++11) std::allocator_traits<Allocator>::const_pointer (since C++11)
iterator	LegacyBidirectionalIterator and ConstexprIterator (since C++26) to value_type
const_iterator	LegacyBidirectionalIterator and ConstexprIterator (since C++26) to const value_type
reverse_iterator	std::reverse_iterator<iterator>
const_reverse_iterator	std::reverse_iterator<const_iterator>

Member functions

(constructor)	constructs the list (public member function)
(destructor)	destructs the list (public member function)
operator=	assigns values to the container (public member function)
assign	assigns values to the container (public member function)
assign_range (C++23)	assigns a range of values to the container (public member function)

get_allocator	returns the associated allocator (public member function)
Element access	
front	access the first element (public member function)
back	access the last element (public member function)
Iterators	
begin cbegin (C++11)	returns an iterator to the beginning (public member function)
end cend (C++11)	returns an iterator to the end (public member function)
rbegin crbegin (C++11)	returns a reverse iterator to the beginning (public member function)
rend crend (C++11)	returns a reverse iterator to the end (public member function)
Capacity	
empty	checks whether the container is empty (public member function)
size	returns the number of elements (public member function)
max_size	returns the maximum possible number of elements (public member function)
Modifiers	
clear	clears the contents (public member function)
insert	inserts elements (public member function)
insert_range (C++23)	inserts a range of elements (public member function)
emplace (C++11)	constructs element in-place (public member function)
erase	erases elements (public member function)
push_back	adds an element to the end (public member function)
emplace_back (C++11)	constructs an element in-place at the end (public member function)
append_range (C++23)	adds a range of elements to the end (public member function)
pop_back	removes the last element (public member function)
push_front	inserts an element to the beginning (public member function)
emplace_front (C++11)	constructs an element in-place at the beginning (public member function)
prepend_range (C++23)	adds a range of elements to the beginning (public member function)
pop_front	removes the first element (public member function)
resize	changes the number of elements stored (public member function)
swap	swaps the contents (public member function)
Operations	
merge	merges two sorted lists (public member function)
splice	moves elements from another list (public member function)
remove remove_if	removes elements satisfying specific criteria (public member function)
reverse	reverses the order of the elements (public member function)
unique	removes consecutive duplicate elements (public member function)
sort	sorts the elements (public member function)

Non-member functions	
operator== operator!= (removed in C++20) operator< (removed in C++20) operator<= (removed in C++20) operator> (removed in C++20) operator>= (removed in C++20) operator<=> (C++20)	lexicographically compares the values of two lists (function template)
std::swap (std::list)	specializes the std::swap algorithm (function template)
erase (std::list) erase_if (std::list) (C++20)	erases all elements satisfying specific criteria (function template)

Deduction guides (since C++17)

Notes	
Feature-test macro	Value Std Feature
<code>_cpp_lib_containers_ranges</code>	202202L (C++23) Ranges construction and insertion for containers

__cpp_lib_constexpr_containers 202502L (C++26) constexpr std::list

Example

Run this code

```
#include <algorithm>
#include <iostream>
#include <list>

int main()
{
    // Create a list containing integers
    std::list<int> l = {7, 5, 16, 8};

    // Add an integer to the front of the list
    l.push_front(25);
    // Add an integer to the back of the list
    l.push_back(13);

    // Insert an integer before 16 by searching
    auto it = std::find(l.begin(), l.end(), 16);
    if (it != l.end())
        l.insert(it, 42);

    // Print out the list
    std::cout << "l = { ";
    for (int n : l)
        std::cout << n << ", ";
    std::cout << "};\n";
}
```

Output:

l = { 25, 7, 5, 42, 16, 8, 13, };

Defect reports

The following behavior-changing defect reports were applied retroactively to previously published C++ standards.

DR	Applied to	Behavior as published	Correct behavior
LWG 230 (https://cplusplus.github.io/LWG/issue230)	C++98	τ was not required to be <i>CopyConstructible</i> (an element of type τ might not be able to be constructed)	τ is also required to be <i>CopyConstructible</i>
LWG 276 (https://cplusplus.github.io/LWG/issue276)	C++98	τ was always required to be <i>CopyAssignable</i>	only required if <code>operator=</code> or <code>assign</code> is instantiated with τ

See also

`forward_list` (C++11) [singly-linked list](#)
(class template)